PROYECTO FINAL

As bajo la manga

Sistema web de reservas para barbería

SAL-301 Algoritmos y Programación · UTEPSA

Defensa final · 26/10/2026

C# · ASP.NET Core MVC · PostgreSQL (Supabase)

El problema

- Las citas se coordinan por **WhatsApp, llamadas o un cuaderno**.
- **Choques de horario** y dobles reservas: no se considera cuánto dura cada servicio.
- Citas perdidas u olvidadas: no están en un solo lugar.
- El cliente solo reserva cuando alguien le responde.
- Sin confirmación para el cliente ni control de lo facturado.

Objetivos

> **General:** desarrollar en C# (ASP.NET Core, POO) un sistema web para reservar citas sin choques de horario y gestionarlas desde un panel con roles, con base de datos y confirmación por correo.

- Analizar y diseñar: requerimientos, E-P-S, algoritmos, pseudocódigo, diagramas de flujo y UML.
- Implementar las clases con encapsulamiento y constructores en 2 proyectos (lógica + presentación).
- Reserva en línea que muestre solo los horarios libres según duración y peluquero.
- Panel con roles Administrador y Recepcionista: registrar, reprogramar, cancelar, atender.
- Base de datos en la nube, correo de confirmación, reportes .txt, validaciones y excepciones.
- Publicar, probar y corregir.

Alcance y usuarios

Incluye

- Reserva en 5 pasos: servicio → **peluquero** → fecha y hora → datos → confirmar
- Correo de confirmación (Brevo)
- Panel: agenda, reservas manuales, clientes, servicios, peluqueros, usuarios
- Reportes y exportación a **.txt**; registro de errores en .txt

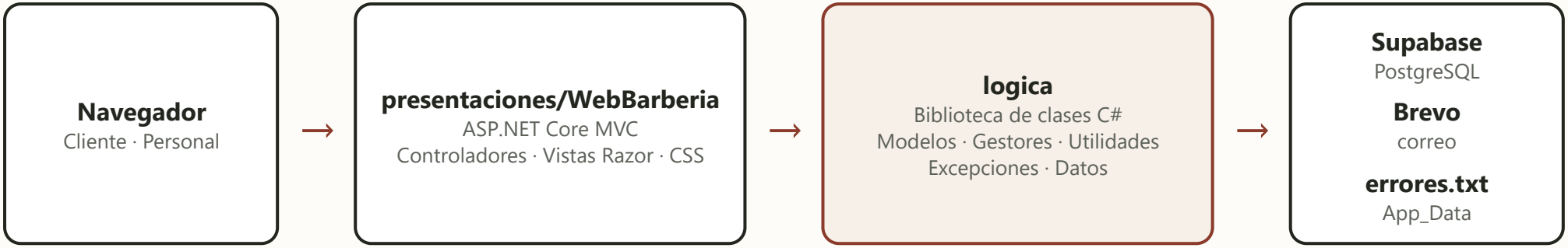
No incluye

- Pagos en línea, app móvil, WhatsApp/SMS
- Sucursales y horarios distintos por peluquero

Usuarios

Actor	Qué hace
Cliente	Reserva desde la web, sin cuenta
Recepcionista	Agenda, reservas manuales, clientes
Administrador	Todo lo anterior + servicios, peluqueros, usuarios, reportes, errores

Arquitectura de la solución



- Dos proyectos, como en clase: la **lógica** no depende de la web y se prueba sola (**Logica.Pruebas**, xUnit).
- Publicado en **MonsterASP.NET**. Las claves no están en el código ni en GitHub.
- Hora oficial **Reloj.AhoraBolivia()** = UTC − 4 (el servidor está en Europa).

Requerimientos y E-P-S

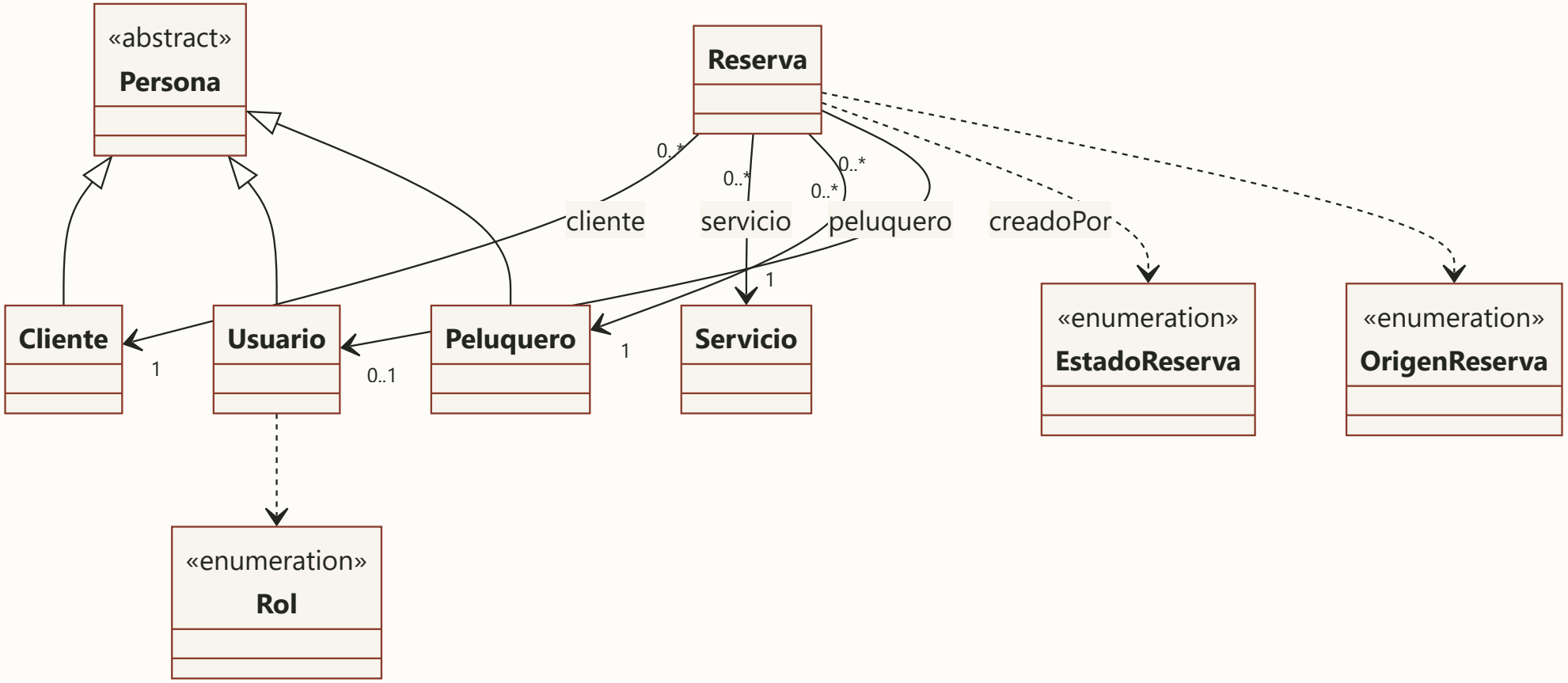
Grupo	Requerimientos funcionales
Cliente	RF-01 a RF-05, RF-24 (catálogo, peluquero, horarios, datos, resumen)
Sistema	RF-06, RF-07, RF-09, RF-22 (registrar, correo, bloqueo, errores)
Personal	RF-08, RF-10 a RF-17, RF-21 (sesión, agenda, reservas, clientes, .txt)
Administrador	RF-18 a RF-20, RF-23, RF-25 (servicios, usuarios, reportes, errores, peluqueros)

25 funcionales · 12 no funcionales · 21 reglas de negocio

Entrada	Proceso (P-02)	Salida
Fecha, servicio, peluquero, reserva a excluir, origen	Validar fecha; recorrer bloques de 30 min; descartar los pasados y los que chocan (A1, A1.1)	Lista de horarios libres

15 procesos E-P-S (P-01 a P-15); cada uno se convirtió en un método.

Diagrama de clases — dominio



POO aplicada

Concepto	En el proyecto	Dónde mostrarlo
Clase y objeto	Reserva es el molde; `new Reserva(...)` crea una cita concreta	**GestorReservas** · GestorReservas.cs · línea 273
Encapsulamiento	Atributos `private`, propiedades de solo lectura; el estado cambia solo con métodos que validan	**Reserva.Cancelar** · Reserva.cs · línea 79
Herencia y abstracción	Persona (abstracta) → Cliente, Usuario, Peluquero	**Persona** · Persona.cs · línea 6
Constructores	Validan al crear (Servicio: duración múltiplo de 30)	**Servicio()** · Servicio.cs · línea 14
Polimorfismo (sobrecarga)	2 versiones de CalcularHorariosDisponibles y 2 constructores de Reserva	**Reserva()** · Reserva.cs · línea 32
Excepción propia	HorarioNoDisponibleException hereda de Exception	**HorarioNoDisponibleException** · HorarioNoDisponibleException.cs · línea 4

Algoritmo central A1 — horarios disponibles

```
Funcion disponibles <- CalcularHorariosDisponibles(F, S, P,
reservasDelDia, idExcluir, origen, ahora)
    Si F >= hoy Y F <= hoy + 30 dias Y F no es domingo Entonces
        d <- S.DuracionMin
        limite <- (origen = Web) ? ahora + 30 min : inicio del
bloque en curso
        R <- reservas de F del peluquero P que ocupan horario
(sin idExcluir)
        Para Cada turno T Hacer
            h <- T.Inicio
            Mientras h + d <= T.Fin Hacer
                Si F + h >= limite Y NO HayChoque(h, d, R)
Entonces Agregar(disponibles, h)
                h <- h + 30 minutos
            FinMientras
        FinPara
    FinSi
FinFuncion
```

- Recibe **ahora** como parámetro → se prueba con una hora fija.
- Solo cuenta las reservas **del peluquero elegido** (RN-21).
- `h + d <= T.Fin`: el servicio entra completo en el turno (no cruza las 12:00).
- **Complejidad O(b · r)**: b ≤ 18 bloques × r reservas del peluquero.

> **GestorReservas.CalcularHorariosDisponibles** · GestorReservas.cs · línea 34

A1.1 HayChoque y prueba de escritorio

```
Funcion choca <- HayChoque(h, d, R)
    fin <- h + d;  choca <- Falso;  i <- 0
    Mientras i < Longitud(R) Y NO choca Hacer
        Si h < R[i].HoraFin Y fin > R[i].HoraInicio Entonces
            choca <- Verdadero
        i <- i + 1
    FinMientras
FinFuncion
```

Intervalos semiabiertos [inicio, fin): 10:00–10:30 y 10:30–11:00 **no** chocan.

GestorReservas.HayChoque · GestorReservas.cs · línea 71

h	h+60	¿Choca?	
09:00	10:00	No	✓
09:30–10:30	…	Sí (10:00–11:00)	✗
11:00	12:00	No	✓
14:00	15:00	No	✓
14:30–16:00	…	Sí (15:00–16:30)	✗
16:30 a 19:00	…20:00	No	✓ ×6

Corte + barba (60 min), Luis, reservas 10:00–11:00 y 15:00–16:30 → **9 horarios**. Coincide con la prueba automática.

Sin choques en dos niveles

1. En C# (A2)

- Valida los datos (Validador).
- **Vuelve a calcular A1** antes de guardar.
- Cliente y reserva en una **transacción**.
- Si el horario ya no está: **HorarioNoDisponibleException**.

GestorReservas.RegistrarReserva · GestorReservas.cs · línea 88

2. En PostgreSQL

```
EXCLUDE USING gist (
  peluquero_id WITH =,
  tsrange(fecha + hora_inicio,
          fecha + hora_fin, '[)') WITH &&
) WHERE estado IN ('Confirmada','Atendida')
```

- Mismo peluquero + rangos que se cruzan → la base rechaza con **23P01**.
- C# lo atrapa con `catch … when` y lanza la excepción propia.

catch PostgresException when 23P01 · GestorReservas.cs · línea 103

CRUD, archivos de texto, validaciones y excepciones

Contenido del módulo	Cómo se cumple	Dónde
CRUD	Clientes, Servicios, Peluqueros, Usuarios y Reservas	**GestorClientes.Registrar** · GestorClientes.cs · línea 58
Archivo .txt (escribir)	Cada error es una línea en App_Data/errores.txt	**RegistroErrores.Registrar** · RegistroErrores.cs · línea 25
Archivo .txt (leer)	El panel muestra las últimas 100 líneas	**RegistroErrores.LeerUltimas** · RegistroErrores.cs · línea 43
Exportar .txt	Agenda del día y reporte	**GestorReservas.AgendaATexto** · GestorReservas.cs · línea 172
Validaciones	Nombre, teléfono (6/7 + 8 dígitos), correo, notas, peluquero	**Validador.ValidarDatosReserva** · Validador.cs · línea 47
Excepciones	try/catch de lo específico a lo general en cada controlador	**ReservarController.Confirmar** · ReservarController.cs · línea 129
Menú principal	Menú lateral del panel según el rol	**_LayoutPanel** · _LayoutPanel.cshtml · línea 15

Reportes: burbuja y búsqueda lineal

Ordenamiento burbuja (A5.1)

```
Para i <- 0 Hasta k - 2 Hacer
    Para j <- 0 Hasta k - 2 - i Hacer
        Si lista[j].Cantidad < lista[j+1].Cantidad Entonces
            Intercambiar(lista[j], lista[j+1])
```

Servicios del más al menos pedido. **O(k^2)**, con k = 5–6 servicios.

GeneradorReportes.OrdenarBurbuja · GeneradorReportes.cs · línea 54

Búsqueda lineal de clientes

Recorre la lista con `foreach` y compara parte del nombre o del teléfono. **O(n)**.

GestorClientes.BuscarEnLista · GestorClientes.cs · línea 22

El reporte (A5)

Un recorrido de las reservas del rango: citas por día, estado y peluquero; ingreso real y estimado. **O(n + k^2)**.

Pruebas

33

pruebas unitarias xUnit, **todas correctas**

- Prueba de escritorio de A1 automatizada
- Choques, descanso, fechas no válidas, margen de 30 min
- Reglas: cancelar / atender, precio copiado, teléfono, correo
- Contraseñas y bloqueo de 3 intentos
- Peluqueros: otro peluquero no choca; reporte por peluquero

ReservaDeOtroPeluquero_NoChoca · HorariosDisponiblesPruebas.cs · línea 48

Algoritmo	Complejidad
A1 Horarios	$O(b \cdot r)$
A1.1 HayChoque	$O(r)$
A3 Iniciar sesión	$O(1)$ + hash
A5 Reporte	$O(n + k^2)$
A5.1 Burbuja	$O(k^2)$
Búsqueda de clientes	$O(n)$

16 casos funcionales con la base de datos documentados en PRUEBAS.md.

Demostración

Web pública

1. Servicio → **peluquero** → fecha (bloques tachados)
2. Otro peluquero: la misma hora libre
3. Datos con errores → validación
4. Resumen → Confirmar → «¡Cita confirmada!» + correo

Panel

5. Iniciar sesión → menú según el rol
6. Agenda: filtrar por peluquero → Exportar .txt
7. Reserva manual · Atendida (antes de hora: no deja) · Reprogramar
8. Clientes: búsqueda e historial
9. Peluqueros y servicios (desactivar)
10. Reportes → Exportar .txt · Registro de errores

Plan B sin internet: Visual Studio → Explorador de pruebas → Ejecutar todas.

Conclusiones

- El sistema **resuelve los choques de horario** en dos niveles: el algoritmo A1 y la base de datos.
- Aplica todos los contenidos del módulo: **POO**, estructuras de control, **CRUD**, **archivos de texto**, **validaciones**, **excepciones**, UML y complejidad.
- Cada peluquero tiene su agenda y el cliente elige con quién atenderse.
- Está publicado y probado: 33 pruebas unitarias y 16 casos funcionales.

Mejoras futuras

- Horarios distintos por peluquero
- Recordatorios por WhatsApp
- Cancelar desde el correo
- Pagos en línea

PROYECTO FINAL

¡Gracias!

Preguntas